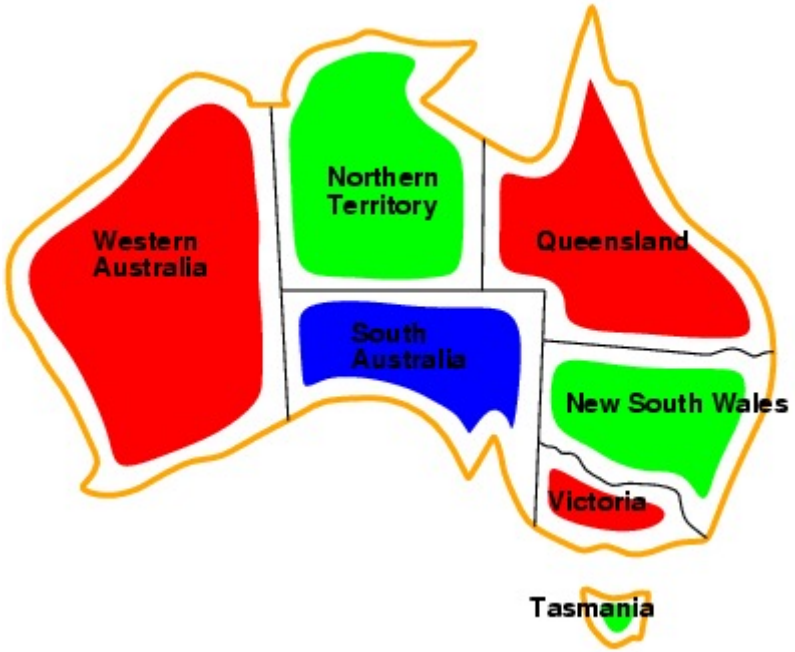
Constraint Solving Problems

Maciej Piasecki

Constraint Solving Problems to be solved by searching

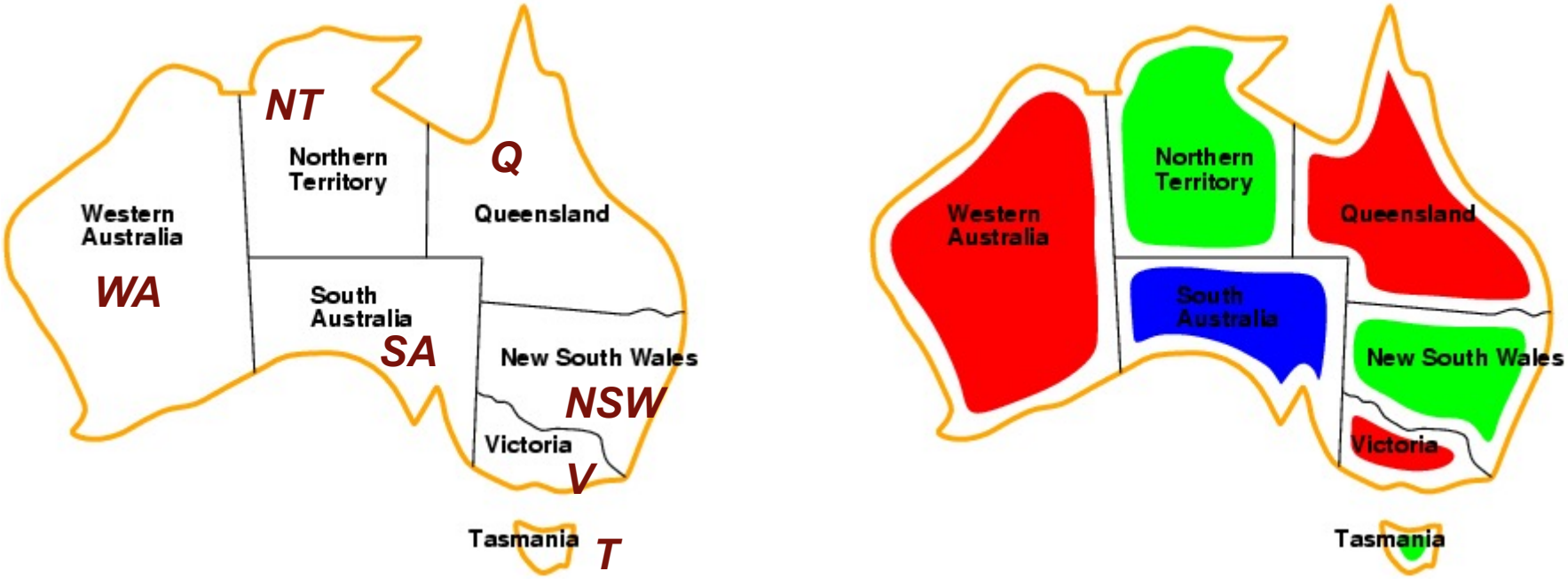
- Another name: Constraint Satisfaction Problems
- Problem: to find appropriate values for a set of variables — a kind of 'puzzles'
- Variables are interdependent — linked by constraints
  - e.g. Australia map colouring to



Searching: Constraint Solving Problems

- Problem characteristics
  - state = set of variables and their values
  - goal = constraint set – of relational form, defined on variables
  - solution = assignment of values to variables in a way satisfying all constraints of the goal
    - operator: value assignment to a variable, a large branching factor
    - e.g., the eight queens chess problem,
- Constraints
  - any relational form (including functional constraints, especially for preferred ones)
  - absolute – an integral part of the goal
  - preferred – introduce a partial order of solutions
- Variables
  - domain (most general constraint)
    - discrete or real value (complicated algebra)
- General rule for uninformed (blind) CSP
  - goal: decomposition into constraint sets for different variables
    - reduction of the very large branching factor
    - determined order

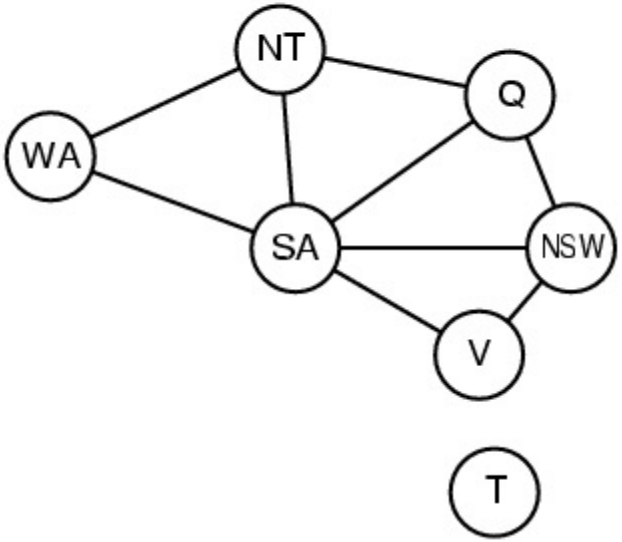
Example



- Variables: *WA, NT, Q, NSW, V, SA, T*
- Domains: D_i = {*red*, *green*, blue}
- Constraints: adjacent regions must have different colours, i.e., WA ≠ NT, or (WA,NT) {(*red*, *green*),(*red*, *blue*),(*green*, *red*), (*green*, *blue*),(*blue*, *red*),(*blue*, *green*)}

CSP as a graph of constraints

- Binary constraints – binary relations
- Graph of constraint:
  - variables – nodes of the graph
  - constraints – arcs of the graph (unlabelled by relation names and undirected)



Searching: Constraint Solving Problems

- Blind algorithms
  - standard – branching factor problem
  - backtracking search
    - backtracking when the constraints are not fulfilled for the present state
- Searching vs constraint propagation and *vice versa*:
  - not all values are worth testing, after some assignments have been done
  - forward checking: removing inconsistent (according to constraints) values from the domains

CSP: Blind searching

function BACKTRACKING-SEARCH(csp) **returns** a solution, or failure
return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment,csp)
returns a solution, or failure

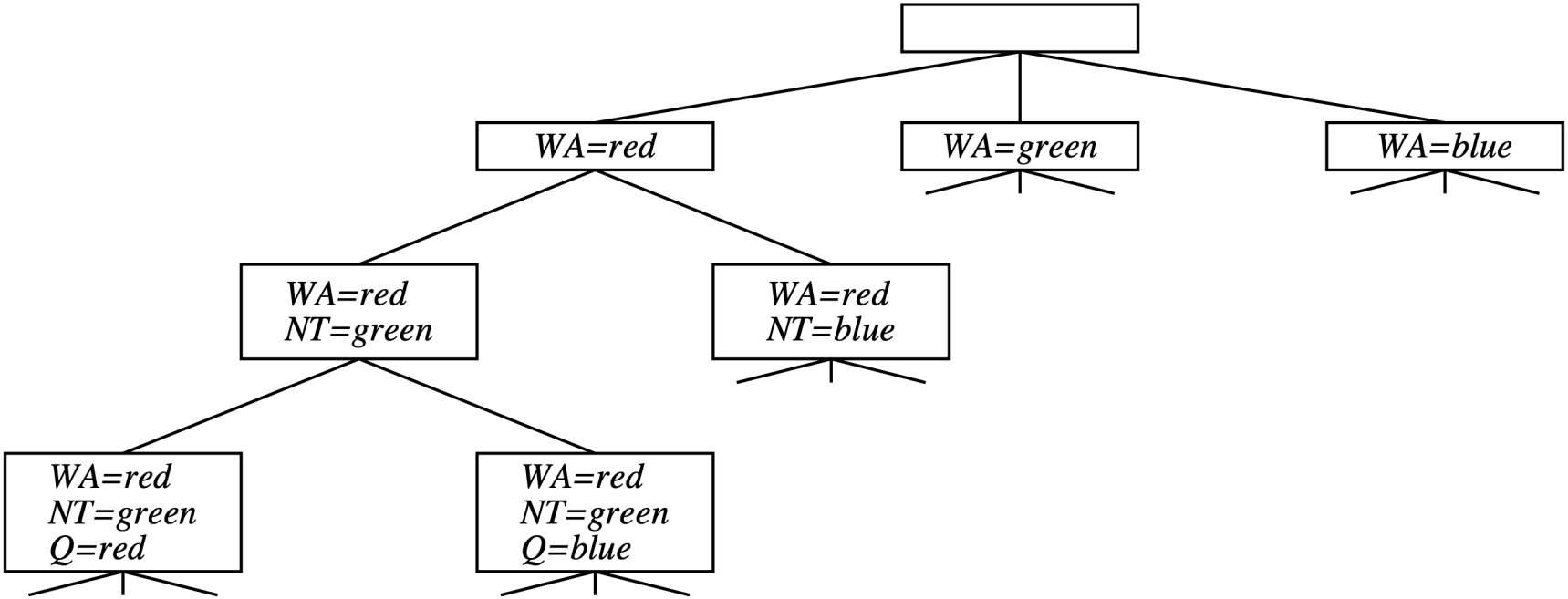
if assignment is complete **then return** assignment
var ←SELECT-UNASSIGNED-
VARIABLE(VARIABLES[csp],assignment,csp)
for each value **in** ORDER-DOMAIN-VALUES(var,assignment,csp) **do**

if value is consistent with assignment according to CONSTRAINTS[csp]
then add {var = value} to assignment
result ← RECURSIVE-BACKTRACKING(assignment, csp)
if result/= failure **then return** result

remove {var = value} from assignment **return** failure

(Russel & Norvig, 1995)

CSP: Blind searching - example



Backtracking search for the map colouring problem.
(Russel and Norvig, Fig. 5.4, 1995)

CSP: Blind searching

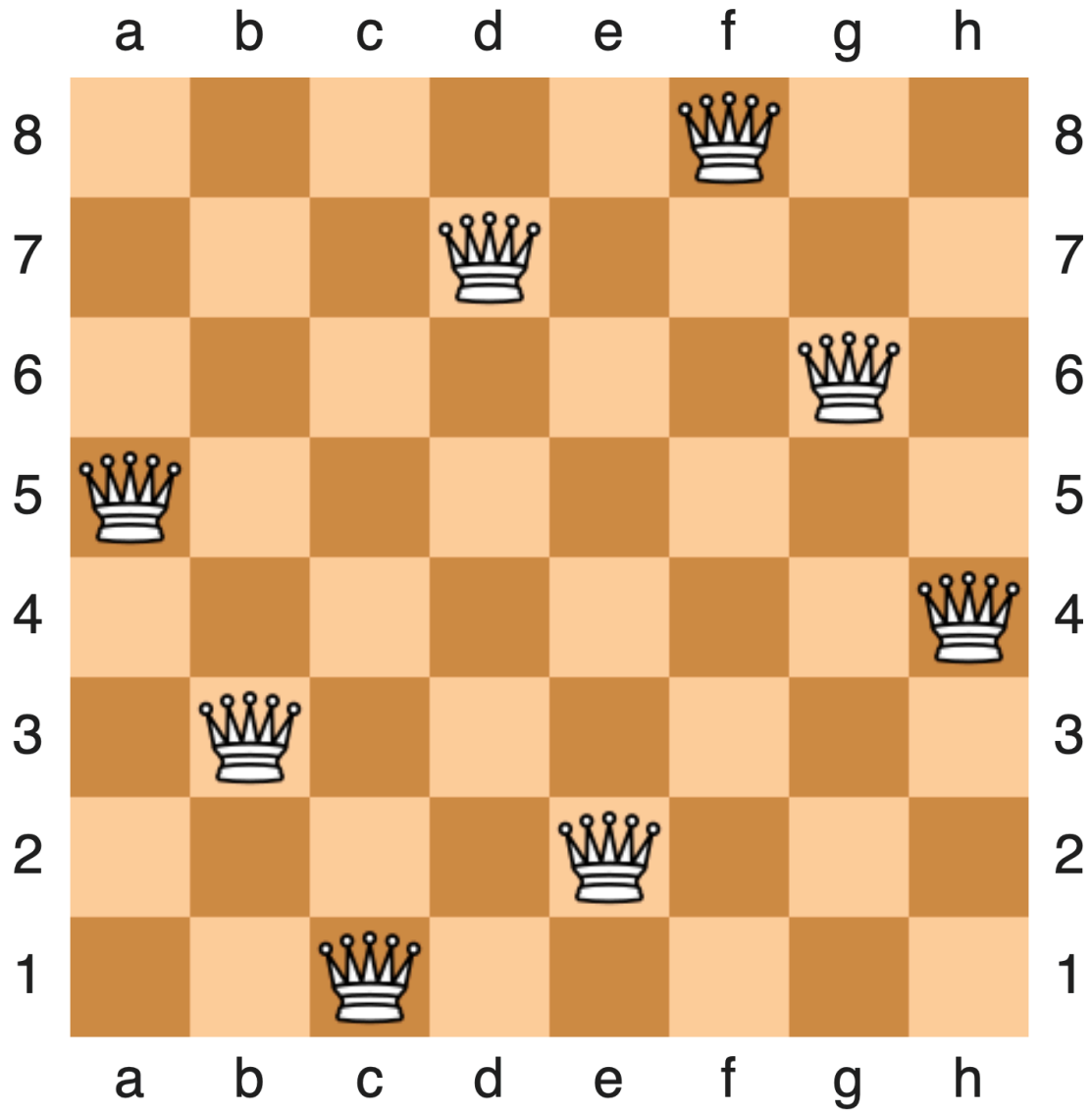
- Searching vs constraint propagation and *vice versa*:
  - not all values are worth testing, after some assignments have been done
  - after each selection of a value for a variable, some values of the other variables do not make sense
  - forward checking: removing inconsistent (according to the constraints) values from the domains

	WA	*NT*	*Q*	*NSW*	*V*	*SA*	*T*
Initial domains	R G B	R G B	R G B	R G B	R G B	R G B	R G B
After *WA=red*	(R)	G B	R G B	R G B	R G B	G B	R G B
After *Q=green*	(R)	B	(G)	R B	R G B	B	R G B
After *V=blue*	(R)	B	(G)	R	(B)		R G B

(Russel & Norvig, edition, 2003)

- Advance constraint propagation:
  - arc consistency: maintaining values in domains that are not contradictory with the constraints – a constraint propagation case
  - higher order consistency — across more arcs, involving more variables

Eight queens puzzle - a solution



Searching: Constraint Solving Problems

- Heuristic algorithms
  - heuristic-based selection of the order of variables and values
  - most-constrained-variable heuristic
    - together with forward checking
    - for the n-queens problem improves from 30 → 100
  - most-constraining-variable heuristic
    - as above, high reduction of the branching factor
  - least-constraining-value heuristic
    - for the n-queens problem improves from 100 → 1000

Constraint Programming: problem definition

- Constraint Satisfaction Problems
  - consistent assignment:
    - does not contradict any of the constraints
  - complete assignment
    - assigns a value for each variable
  - objective function
    - an additional requirement measures the quality of a solution
  - constraint graph:
    - nodes – variables, arcs – constraints (instances)
  - classes
    - discrete domains (finite)
      - worst complexity: $O(d^n)$, where d — the greatest range, n — number of variables
    - infinite domains (discrete, real values)
      - constraint language — a logical or algebraic language
      - range limitation for variables
      - solvable for linear constraints and the integer domain
      - unsolvable for non-linear constraints

CSP: basic division

- Constraints:
  - unary, binary, ...
  - a greater number of arguments may be reduced to binary constraints with the help of additional variables
  - division according to the goal definition:
    - absolute
    - preferred, e.g. higher cost, optimisation problems
- Searching
  - blind: ‚naive', backtracking searching
  - heuristics
    - propagation
    - assignment selection (distribution), selection of the backtracking target
- Propagation
  - forward checking
    - after `assignment(X)`, for each Y connected to X, all values incompatible with the current X value are deleted from the Y domain
    - related to the gathering of data for the Most Constrained Variable heuristics (and also Minimum Remaining Values)
  - constraint propagation

CSP: Algorithms AC-3 i MAC

```
function AC-3(csp) returns false if an inconsistency is found
                       and true otherwise
  inputs: csp, a binary CSP with components (X, D, C)
  local variables: queue – a queue of arcs, initially all the arcs in csp
  while queue is not empty do
        (Xi, Xj)←REMOVE-FIRST(queue)
        if REVISE(csp, Xi, Xj) then
              if size of Di = 0 then return false
              for each Xk in Xi.NEIGHBORS - {Xj} do
                    add (Xk, Xi) to queue
  return true
```

(Russel & Norvig, edition, 2003, p. 146)

CSP: Algorithms AC-3 i MAC

```
function REVISE(csp, Xi, Xj) returns true iff we revise the domain
of Xi     revised ← false
          for each x in Di do
                    if no value y in Dj allows (x,y) to satisfy
                              the constraint between Xi and Xj then
                              delete x from Di
                              revised ← true
          return revised
```

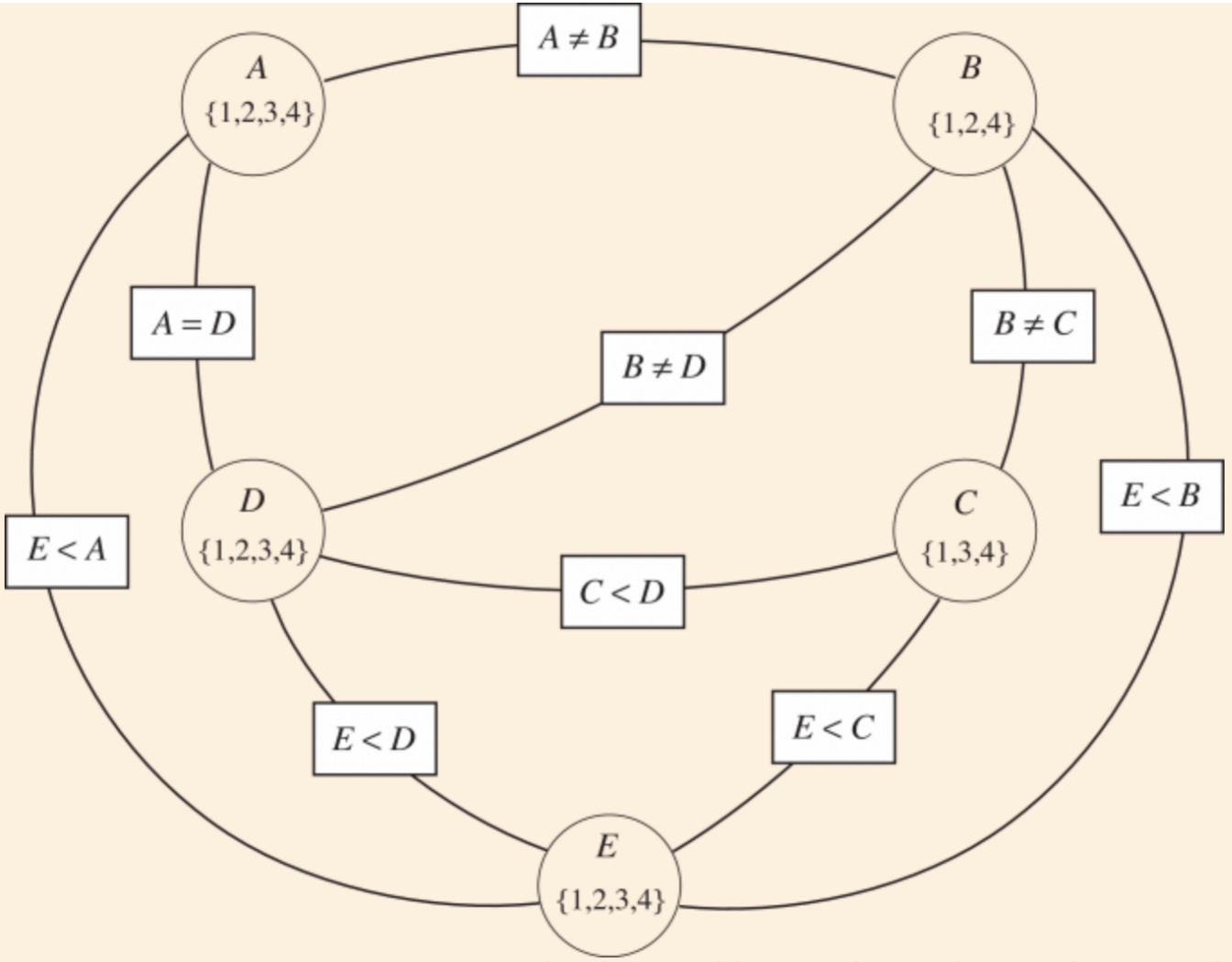
- Maintaining Arc Consistency (MAC)
  - after a variable **Xi** is assigned a value (during searching)
  - AC-3 is called for a **Xj** variable which is connected to **Xi** and is not assigned a value yet
    - any link will start propagation for the whole graph

(Russel & Norvig, edition, 2003, p. 146)

Constraint Propagation: examples

- Example 1:
  - $\{X \in [23, 100], Y \in [1,33], X < Y, Y > X\}$
  - $X<Y \Rightarrow_{\text{Prop}} \{X \in [23, 32], Y \in [1,33]\}$
  - $Y > X \Rightarrow_{\text{Prop}} \{X \in [23, 32], Y \in [24, 33]\}$
  - the next two iterations do not change anything in the domains
- Example 2:
  - $\{X \in 0\#9, Y \in 0\#9\}\ X + Y = 9, 2*X + 4*Y = 24$
  - constraint 2: $\{X \in [0, 8], Y \in [2, 6]\}$
  - constraint 1: $\{X \in [3, 7], Y \in [2, 6]\}$
  - constraint 2: $\{X \in [4, 6], Y \in [3, 4]\}$
  - constraint 1: $\{X \in [5, 6], Y \in [3, 4]\}$
  - constraint 2: $\{X = \{6\}, Y = \{3\}\}$
- Example: stable propagators that do not determine a solution
  - $\{X \in [4, 10], Y \in [1, 7], Z \in [3, 9]\}\ 3*X + 3*Y = 5*Z, X-Y=Z, X+Y=Z+2$

Constraint Propagation: example – an unstable graph



- http://artint.info/2e/html/ArtInt2e.Ch4.S4.html

CSP: propagation and distribution

- Distribution heuristics
  - the order in which different values are tested during searching makes a difference
  - Minimum Remaining Values = Most Constrained Variable = First Fail
  - Degree Heuristic = Most Constraining Variable
  - Least-constraining Value
    - often applied in combination with AC-3
    - and tracking of changes in the supporting sets or conflict sets
  - domain splitting
    - mostly binary divining
    - maximum, minimum first
    - heuristic split

CSP: propagation

- k-consistency
  - CSP is k-consistent iff for each set of k-1 variables, and for each consistent assignment to these variables, a consistent value may be assigned to the k-th variable
    - 2-consistency = arc consistency,
    - 3-consistency = path consistency
  - strongly k-consistent constraint graph:
    - k-consistent, k-1 consistent, ...
    - guarantees finding a solution with the complexity O(nd)
  - each algorithm for maintaining n-consistency has exponential complexity
- Constraints of special types, e.g.
  - alldiff — different value of all variables
    - an efficient algorithm for checking the constraint
  - atmost — resource constraint (total sum)
    - e.g. atmost(10;PA1;PA2;PA3;PA4)
- Registration of the propagation results
  - domain propagation
  - bounds propagation
    - recording and modifying the lower and upper bounds

CSP: heuristic search

- Search strategies
  - 'naive' chronological backtracking
  - backjumping method
    - idea:
      - returning to one of the variables that ‚eliminated' values in the variable that causes backtracking
    - conflict set (direct)
      - let X be a node that reveals inconsistency,
      - CS (conflict set) — a set of nodes linked by constraints with X
    - action: return to Y∈CS
    - building CS during forward checking
    - redundant to forward checking
  - conflict-directed backjumping
    - idea:
      - returning to the reason for a failure
    - indirect (deep) conflict set — redefined:
      - let: Xj is the current variable, conf(Xj) is its direct CS, Xi∈conf(Xj) directly precedes Xj
      - conf(Xi) = conf(Xi) ∪ conf(Xj) \ {Xi}
    - constraint learning

CSP: examples of problems

Example 4.9. *Suppose the delivery robot must carry out a number of delivery activities, a, b, c, d, and e. Suppose that each activity happens at any of times 1, 2, 3, or 4. Let A be the variable representing the time that activity a will occur, and similarly for the other activities. The variable domains, which represent possible times for each of the deliveries, are*

$$dom(A) = \{1,2,3,4\}, \quad dom(B) = \{1,2,3,4\}, \quad dom(C) = \{1,2,3,4\},$$
$$dom(D) = \{1,2,3,4\}, \quad dom(E) = \{1,2,3,4\}.$$

Suppose the following constraints must be satisfied:

$$\{ \quad (B \neq 3),\ (C \neq 2),\ (A \neq B),\ (B \neq C),\ (C < D),\ (A = D),$$
$$(E < A),\ (E < B),\ (E < C),\ (E < D),\ (B \neq D)\}$$

It is instructive for you to try to find a model for this example; try to assign a value to each variable that satisfies these constraints.

http://artint.info/2e/html/ArtInt2e.Ch4.S1.SS3.html

CSP: full state paradigm

- Idea of local search in CSP
  - searching based on a complete state specification
  - least constraining value heuristic
    - efficiency depends on the selection of the initial state
  - examples:
    - for the 1 million queens problem it gives a solution in 50 steps
    - scheduling tasks for the Hubble Space Telescope: from 3 weeks to 10 minutes!
  - ability to generate a new solution for changed conditions

function MIN-CONFLICTS(csp, max_steps) **returns** a solution or failure
inputs: csp, a constraint satisfaction problem
max_steps, the number of steps allowed before giving up

$current \leftarrow$ an initial complete assignment for csp
for $i = 1$ to max_steps **do**
if $current$ is a solution for csp **then return** $current$
$var \leftarrow$ a randomly chosen, conflicted variable from VARIABLES[csp]
$value \leftarrow$ the value v for var that minimizes CONFLICTS($var, v, current, csp$)
set $var = value$ in $current$
return $failure$

CSP: exploring the problem structure

- Problem structure vs solution method
  - independent subproblems
    - disconnected graphs components (but internally connected)
    - complete solution as a sum of solutions for all graph components
  - tree-shaped constraint graph
    - „each CSP of a tree structure may be solved in time linearly dependent on the number of variables”
      - (Russel i Norvig, 2003)
      - algorithm for a tree structured CSP pp. 152, (‘new’ Chapter 5) — $O(nd^2)$
  - reduction of a graph into a tree
    - removing nodes
      - setting values for
        - the removed node
        - and the left for the other nodes
      - problem relaxation — to a tree structured
    - collapsing nodes
      - construction of a decomposition tree
        - tree of subgraphs

Literature

- http://artint.info/html/ArtInt_76.html and the following chapters from the online: Artificial Intelligence: Foundations of Computational Agents, Cambridge University Press, 2010: http://artint.info
- Constraint Satisfaction Problems: Algorithms and Applications: Page 25, 1999. http://cepac.cheme.cmu.edu/pasilectures/henning/EJOR-BrailsfordSmith.pdf
- Laveen Kanal, Vipin Kumar (ed.)Search in Artificial Intelligence, Springer, 1988 https://link.springer.com/book/10.1007/978-1-4613-8788-6
- Algorithms for Constraint Satisfaction Problems CSPs. Master thesis, 1998. http://www.cs.toronto.edu/~fbacchus/Papers/liu.pdf
- Stuart Russell and Peter Norvig. Artificial Intelligence: A Modern Approach. 3rd edition, Pearson, 2010.
  - Chapter 5 (a new version): http://aima.cs.berkeley.edu/newchap05.pdf
  - Materials: http://aima.cs.berkeley.edu/index.html